Path Retraceability and the Accountability Vocabulary: What CKS Substrates Must Carry to Remain Auditable

Author: Wenxin Li (Independent Researcher)
ORCID: 0009-0004-8065-3235
Date: 26 April 2026

Attribution

This work derives from and formalizes the Coordination Knowledge Substrate (CKS) pattern introduced in *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems* (Li, April 2026). It does not introduce new axioms. Its sole contribution is to articulate, in operational form, the relationship between two named vocabularies the source paper imports for its traceability commitment — *path retraceability* and the *accountability plan / accountability trace* pair — so that downstream work has a precise specification of what a CKS substrate must carry to remain auditable.

Abstract

Claim 1 of the CKS pattern names traceability and auditability among the constitutive design goals of a CKS substrate (§3.1). To make those design goals operationally precise, the source paper imports two named vocabularies from prior work: *path retraceability* (Rajabi & Kafaie, 2022) and the *accountability plan* / *accountability trace* pair (Naja et al., 2021). Both vocabularies do work across Claims 2–6 without being redefined, but the source paper does not articulate the relationship between them in one place. This note states the relationship. Path retraceability is a structural property of substrate content: every piece of content carries enough provenance that its causal antecedents can be reconstructed by reading substrate content alone. The accountability vocabulary is a contract layer with two coupled halves: the plan specifies what the trace must capture (in CKS terms, the substrate schema and orchestration rules together); the trace records what the plan required (in CKS terms, substrate content as it accumulates, including its provenance metadata). A CKS substrate must satisfy both halves. The note states what each vocabulary names, what each requires of substrate content, what each does not require, what distinguishes the combined commitment from audit logging, version history, and ML-style explainability, and what operational test a system must pass to instantiate the commitment.

1. Why the two vocabularies need to be distinguished

The source paper commits to traceability, auditability, conflict preservation, and governance as the constitutive design goals of a CKS substrate at the coordination and decision layer (§3.1). To specify what those goals require operationally, §3.1 imports two named vocabularies from prior work: path retraceability from Rajabi and Kafaie (2022), and the accountability-plan / accountability-trace distinction from Naja et al. (2021). Both vocabularies do work across Claims 2 through 6 without being redefined; the source paper commits to them once, in §3.1, and then uses them.

What the source paper does not do — and what this note does — is articulate, in one place, the relationship between the two vocabularies. The relationship is what makes the traceability commitment operationally precise, and it is also what distinguishes a CKS substrate from a system that merely logs events. Audit logging records what happened; the CKS traceability commitment requires the substrate carry enough state to reconstruct *why and under what authority* — a stronger and more specific requirement, and one that fails for systems whose audit logs are complete but whose substrate content carries no provenance. Without the relationship between the two vocabularies stated explicitly, downstream implementations are free to satisfy one and ignore the other, or to substitute generic audit logging for both.

2. Path retraceability — the first vocabulary

What it names. Path retraceability is the property that any decision, output, or piece of substrate content can be traced back through a path of antecedent substrate content to the inputs and orchestration rules that produced it. The path is not a log of events external to the substrate; it is a structural property of the substrate itself. A reader can answer "what produced this?" by reading substrate content alone, without consulting external logs, agent memory, or human recollection.

What it requires. For path retraceability to hold, every piece of substrate content must carry sufficient provenance that the path back to its antecedents is reconstructable from substrate content alone. The provenance fields include the writer (a human, or an LLM operating under a named orchestration rule in a named cell execution), the timestamp, the orchestration rule under which the content was written (if the writer was a cell), and the antecedent substrate content the writer drew on. With these fields present, every step of the path is itself a substrate read; without them, the path runs through information the substrate does not carry, and retraceability fails.

What it does not require. Path retraceability does not require that every cell execution be replayable in a deterministic-execution sense. LLM outputs are non-deterministic; what is retraceable is the substrate path, not the LLM's internal reasoning. The architecture does not attempt to make the LLM's internal reasoning retraceable, and path retraceability does not depend on it doing so. Path retraceability also does not require any specific representation format — the path may be carried as graph edges, as referenced cell identifiers, as foreign-key relationships, or as any other addressable form. What it requires is that the path exist as addressable substrate content, not how that content is encoded.

3. The accountability vocabulary — the second vocabulary

The accountability vocabulary, imported from Naja and colleagues (2021) via §3.1, has two coupled halves: the accountability plan and the accountability trace. The two terms name distinct objects, and treating them as a single concept loses the failure modes that distinguishing them makes nameable.

The accountability plan. The accountability plan is the specification of what the system must capture about decisions, behavior, and authority — stated in advance, before specific events occur. It is the contract the system binds itself to before there is any history to record. In CKS terms, the substrate schema and the orchestration rules together constitute the accountability plan. The substrate schema specifies what fields each piece of substrate content must carry — what writer attribution accompanies it, what timestamps are required, what antecedent references must be present, what conflict

relationships must be modeled. The orchestration rules specify what cells must record about their behavior — what rule identifier accompanies a cell-mediated write, what rationale fields the rule requires the cell to populate, under what conditions the rule directs the cell to write what content. Schema and rules together specify, in advance, what the trace must contain to be complete.

The accountability trace. The accountability trace is the recorded history of what actually occurred under the plan: the cell executions that ran, the substrate writes that happened, the contradictions that were preserved, the resolutions that were recorded. In CKS terms, the accountability trace is substrate content as it actually exists at any given moment, including its provenance metadata. Substrate content *is* the trace; the substrate is not merely the medium on which the trace is recorded.

The two are coupled. The plan specifies what the trace must capture; the trace records what the plan required. A CKS substrate that has a trace without a plan can record events but cannot certify that the recording was complete — completeness is what the plan defines, and without a plan there is no standard against which the trace's completeness can be evaluated. A substrate that has a plan without a trace specifies an obligation it cannot demonstrate having met. Both halves are required, and the source paper's commitment to both is what §3.1's traceability commitment names when read at the contract layer.

4. How the two vocabularies relate

Path retraceability and the accountability vocabulary operate at different layers of the same commitment. Stating the relationship precisely is the central work of this note.

Path retraceability is a structural property of substrate content at any given moment: can a reader, given the substrate as it currently stands, reconstruct the path from any piece of content back to its antecedents? It is a property of the *trace* — a property the trace either has or does not have, evaluated on the substrate's current state.

The accountability vocabulary is a *contract layer*. The plan specifies what the trace must support; the trace satisfies the plan, or fails to. Path retraceability is one of the properties the plan can require the trace to support, and in CKS the plan does require it: the substrate schema and the orchestration rules together specify the provenance fields the trace must carry, and those fields are exactly what path retraceability needs.

The two vocabularies together specify a commitment that has two parts: (a) a plan that specifies what must be retraceable, and (b) a trace that actually preserves the retraceable paths. CKS commits to both. The substrate schema and the orchestration rules constitute the plan. The substrate content as it accumulates constitutes the trace. Path retraceability is the structural property the trace must have for the plan to be satisfied. Each vocabulary alone is insufficient — a structural property no contract requires is unenforceable; a contract no realized state satisfies is unmet.

5. What the commitment requires of substrate content

The commitment is operational only to the extent that what the substrate must carry is specified. For each piece of substrate content, the substrate must carry the following:

(a) Writer attribution. Who or what wrote this content — a human acting under preserved override authority, or an LLM operating under a named orchestration rule in a named cell execution. The attribution distinguishes the two cases without collapsing them.

(b) Timestamp. When the content was written. The timestamp is what makes the path orderable; without it, antecedence relationships in the substrate cannot be evaluated as paths over time.

(c) Antecedent reference. What prior substrate content (if any) the writer drew on. For cell-mediated writes, the antecedents are the substrate content the cell read as input. For direct human writes, the antecedents may be the substrate content the human was responding to, or may be empty for original input. Without antecedent references, each piece of substrate content is an island.

(d) Rule reference (where applicable). The orchestration rule under which the content was written, for cell-mediated writes. For direct human writes, the rule reference is the human's exercise of override authority — itself a substrate-recordable fact.

(e) Rationale (where applicable). The reason the writer wrote what they wrote, where the orchestration rule or the substrate schema requires rationale to be captured. Rationale is not required of every piece of content; it is required where the plan requires it.

(f) Relationship to contradicting content (where applicable). For content that contradicts other substrate content, the explicit relationship to the contradicting content, per the conflict-preservation commitment Claim 3 defends and §5.1 develops. The relationship itself is substrate content; it is part of the trace.

The first four — writer attribution, timestamp, antecedent reference, rule reference — are what path retraceability requires, because without them the path back to antecedents is not reconstructable from substrate content alone. The fifth and sixth — rationale and contradiction relationships — are required when the accountability plan calls for them. Together, the six fields make any decision in the system reconstructable from the substrate alone.

6. What the commitment is NOT

Three adjacent commitments are commonly conflated with the CKS traceability commitment, and each conflation produces a different misreading of what the architecture requires.

Not audit logging. An audit log records events external to the substrate — who accessed what, when, from where. Path retraceability records causal antecedence within the substrate — what content led to what content. The two cover different objects. A system can have complete audit logging and still fail path retraceability if the substrate content it records does not carry antecedent references; conversely, a substrate satisfying path retraceability records causal paths an access log cannot, because the access log is about who touched what, not what produced what. CKS requires the latter and is silent on the former; an organization that wants both adopts both, recognizing they are different commitments.

Not version history. Version history records changes to substrate content over time — what each successive state was. Path retraceability records *why* each version exists — under what authority, drawing on what antecedent content, under what orchestration rule. A version-controlled substrate without writer attribution, antecedent references, and rule references satisfies versioning but not

retraceability: the reader can see what changed, but not what produced the change.

Not explainability in the ML sense. Machine-learning explainability is the project of reconstructing why a model produced a specific output — attribution to features, attention patterns, intermediate activations. Path retraceability does not attempt this. The LLM's internal reasoning is not retraceable in the CKS sense, and the architecture does not require it to be. What is retraceable is the substrate path: the cell that executed, the orchestration rule it operated under, the substrate content it read, the substrate content it wrote.

7. How the commitment depends on other commitments

The traceability commitment is defensible because other commitments in the CKS pattern hold; weakening any of them weakens it.

It depends on **substrate-as-source-of-truth** (§11.3). The retraceable path runs through the substrate, so the substrate must be authoritative. A system in which decisions are reached outside the substrate and then summarized into it cannot satisfy path retraceability, because the antecedent path runs through content the substrate does not carry.

It depends on **AI-as-substrate-mediator**. LLM writes are attributable because they happen through cells under orchestration rules, with cell identity and rule identity recorded as substrate content. An LLM operating outside this commitment — writing content directly to the substrate without cell or rule attribution, or writing to surfaces other than the substrate — would break the writer-attribution requirement of §5(a) and the rule-reference requirement of §5(d).

It depends on the **substrate–cell boundary**. Cells communicate through the substrate, which is what makes cell-to-cell paths retraceable as substrate paths. Cell-to-cell direct communication that bypassed the substrate would create antecedence relationships the substrate could not record, and path retraceability would fail across those boundaries.

It depends on **conflict preservation**. Contradictions and their resolutions are themselves traceable substrate content — the relationship to contradicting content (§5(f)) is a field substrate content carries, not an event external to the substrate. If conflicts were resolved silently, the trace would be missing exactly the substrate content the plan requires.

8. Operational test

A system implements the path retraceability and accountability commitments if and only if all of the following are true:

1. Every piece of substrate content carries writer attribution, timestamp, and antecedent reference (where antecedents exist).

2. Cell-mediated writes carry rule references identifying the orchestration rule under which the cell wrote.

3. Where the accountability plan — the substrate schema together with the orchestration rules — calls for rationale, rationale is captured as substrate content.

4. Contradictions carry explicit relationship references to the content they contradict, per the conflict-preservation commitment.

5. Any decision the system has reached can be answered from substrate content alone, without consulting external logs, agent memory, or human recollection.

A system that fails any of (1)–(5) may be a useful system, and may be auditable in some other sense, but does not implement the CKS traceability commitment.

9. Why naming the two vocabularies matters

Each failure mode the relationship between the two vocabularies makes nameable is a real one in current practice. Implementations that conflate path retraceability with audit logging produce systems that record events but cannot reconstruct decisions. Implementations that have a plan without a trace specify obligations they cannot demonstrate having met. Implementations that have a trace without a plan accumulate content without certifying completeness. Implementations that treat path retraceability as a structural property requiring no contract have no standing to compel its presence. Each failure mode is identifiable only because the relationship between the two vocabularies is stated; each is invisible if the vocabularies are collapsed or used interchangeably.

The source paper commits to both vocabularies in §3.1 and uses them across Claims 2 through 6. This note states what their relationship is, so that subsequent work that adopts the CKS pattern, extends it, or composes it with adjacent patterns has a precise specification of what the substrate must carry to satisfy the traceability commitment, and a precise specification of where each adjacent pattern would fail to satisfy it.

Source paper

Li, W. (2026). *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems.* April 2026. ORCID: 0009-0004-8065-3235.

How to cite this note

Li, W. (2026). *Path Retraceability and the Accountability Vocabulary: What CKS Substrates Must Carry to Remain Auditable.* 26 April 2026. ORCID: 0009-0004-8065-3235.